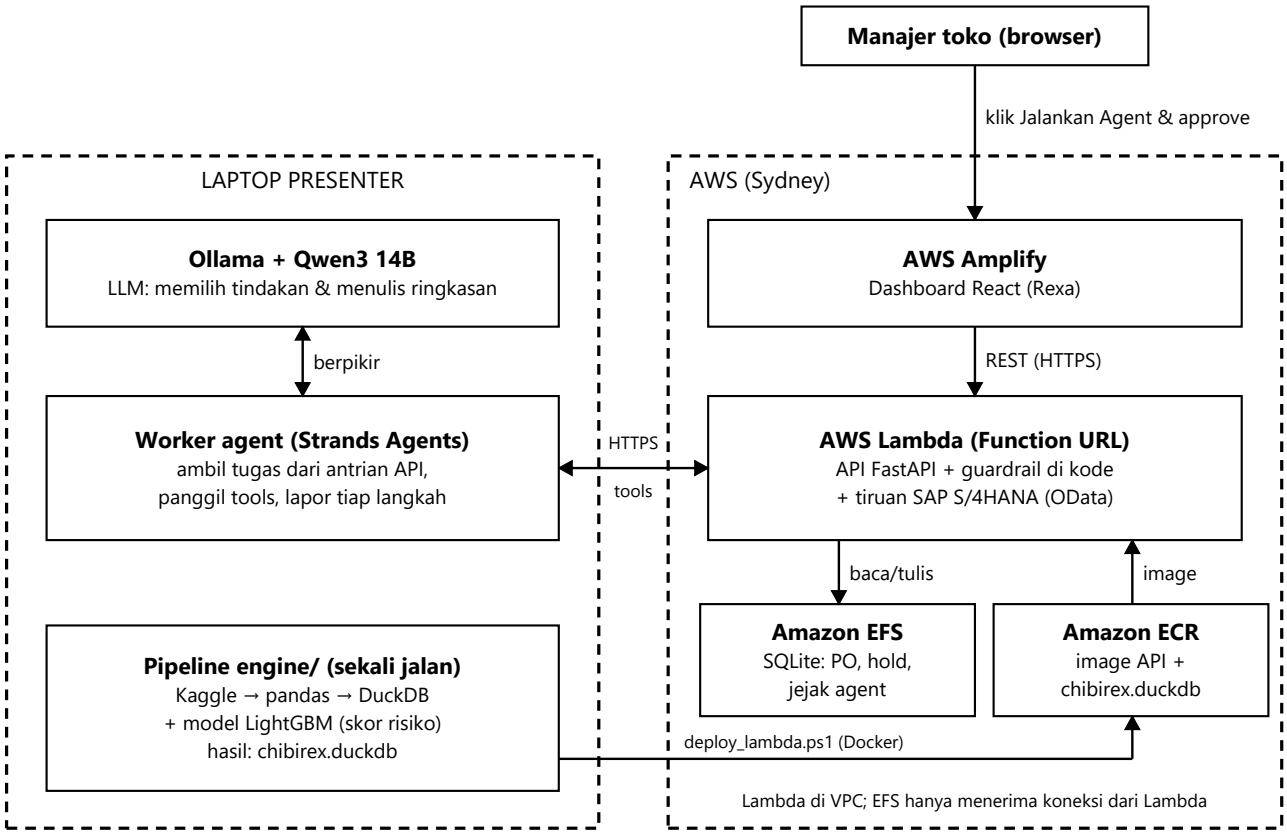
Komponen Teknologi Rexa

Smart Inventory & Cash Flow Copilot · Tim ChibiRex · Agentic AI Hackathon 2026 · kondisi per 22 Oktober 2026

Ringkasnya: agent AI hanya **memilih dan menjelaskan** tindakan; **angka dihitung oleh data** (DuckDB dan model LightGBM), **aturan dijaga oleh kode** (guardrail di API), dan **keputusan akhir tetap di tangan manusia** (1-Click Approval sebelum PO masuk SAP).

Diagram arsitektur



Dashboard, API, guardrail, penyimpanan, dan tiruan SAP berjalan di AWS. Hanya LLM (Ollama) yang berjalan di laptop presenter, lewat worker yang mengambil tugas dari antrian API.

1. Data dan analitik ("bahan" dan "hitungan")

Komponen	Untuk apa	Berjalan di
Kaggle	Sumber dataset. **Bibitor**: stok, penjualan, dan pembelian 80 toko minuman. **Instacart**: isi keranjang belanja, untuk cek substitusi/peran komplementer. **Product Backorders**: melatih model risiko stok habis. **Online Retail II**: pola pesanan borongan untuk menguji guardrail.	Diunduh sekali di laptop
Python + pandas	Bahasa utama proyek dan alat olah data.	Laptop
DuckDB	Database analitik dalam **satu file** (`chibirex.duckdb`, ±300 MB). Menghitung kecepatan jual (Vs), reorder point, safety stock, aturan substitusi, Health Score, dan menyimpan skor risiko. Cepat untuk data besar dan tidak perlu server database.	Dibuat di laptop; ikut di image Lambda (hanya dibaca)
LightGBM	Model machine learning yang memperkirakan SKU berisiko kehabisan stok; sumber **Confidence Score** 0–100 (TINGGI ≥ 90, SEDANG ≥ 70). Di data uji, ±2,9× lebih tepat daripada aturan sederhana.	Dilatih di laptop
Pipeline engine/	Rangkaian langkah `python -m engine.pipeline` yang mengubah data mentah Kaggle menjadi `chibirex.duckdb`.	Laptop

2. API dan guardrail ("aturan main")

Komponen	Untuk apa	Berjalan di
FastAPI	Membuat **REST API**: satu-satunya pintu untuk semua aksi. Berisi 6 tools agent dan endpoint dashboard (daftar toko, Health Score, antrian agent, approval PO).	AWS Lambda (online) atau laptop
Pydantic	Memeriksa setiap data yang masuk; field yang tidak dikenal ditolak (agent tidak bisa menyelipkan `"status": "APPROVED"`).	Di dalam API
Guardrail di kode	Pengaman yang tidak bisa dilewati agent atau prompt (rincian di tabel berikut).	Di dalam API
SQLite	Database kecil untuk data **transaksi**: draf PO, hold, hitung fisik, idempotency key, circuit breaker, audit log, jejak langkah agent, antrian tugas.	File di Amazon EFS (online) atau laptop
Tiruan SAP	Meniru API Purchase Order SAP S/4HANA (OData V2 `API_PURCHASEORDER_PROCESS_SRV`, token CSRF, nomor PO 45xxxxxxxx). PO yang disetujui manajer dikirim ke sini.	Di dalam API
uvicorn	Server untuk menjalankan API di laptop (pengembangan dan cadangan demo).	Laptop

Guardrail di kode

Guardrail	Cara kerja
Filter 3σ	Qty PO di atas target stok + 3σ permintaan dianggap lonjakan tidak wajar (misalnya pesanan borongan) → ditahan.
Batas PO harian	Total nilai PO per toko per hari maksimal Rp100 juta.
Circuit breaker	Kalau 3 PO ditahan guardrail dalam 60 menit, semua PO baru ditahan sampai manajer mereset.
Idempotency key	Request yang diulang dengan kunci yang sama tidak membuat PO atau hold ganda.
Draf secara default	Agent hanya menyusun draf; PO baru masuk SAP setelah manajer menekan 1-Click Approval. Satu draf per SKU.
Data Confidence	Kalau data stok tidak bisa dipercaya (di bawah 0,8, misalnya ada selisih hitung fisik), PO ditahan sampai stok diaudit.
Syarat Hold Reorder	Reorder hanya boleh ditahan kalau penjualan turun dan jenis produknya tidak komplementer; 10% tetap dibeli sebagai stok uji (exploration budget).

3. Agent ("otaknya")

Komponen	Untuk apa	Berjalan di
Strands Agents	SDK open-source dari AWS untuk membuat agent: mengatur putaran LLM berpikir → memanggil tool → membaca hasil → berpikir lagi sampai tugas selesai.	Laptop
Ollama + Qwen3 14B	LLM (model bahasa) yang berjalan di laptop dengan GPU. Tugasnya memilih tindakan dan menulis ringkasan; tidak menghitung angka (semua angka dari API).	Laptop (GPU)
Worker	Jembatan antara dashboard online dan LLM di laptop: mengambil tugas dari antrian API, menjalankan agent, dan melaporkan setiap langkah ke API (`python -m agent.worker`).	Laptop
httpx	Pustaka yang dipakai agent dan worker untuk memanggil API lewat internet.	Laptop
Amazon Bedrock	Rencana awal: LLM di AWS. **Dibatalkan**: kuota token Bedrock di akun tim bernilai 0.	-

6 tools yang boleh dipakai agent (semuanya lewat API, jadi tetap dijaga guardrail)

Tool	Fungsi
list_prioritas	Daftar SKU yang perlu tindakan di satu toko: naik (kandidat draf PO) dan turun (kandidat hold).
get_sku_metrics	Detail satu SKU: stok, Vs, reorder point, saran PO, skor risiko, Data Confidence.
check_substitution	Cek apakah jenis produk punya peran komplementer (dari data keranjang Instacart).
hold_reorder	Tahan reorder SKU yang penjualannya turun dan tidak komplementer; menghitung modal yang diselamatkan.
suggest_reallocation	Saran pengalihan modal dari reorder yang ditahan ke SKU yang penjualannya naik.
draft_purchase_order	Susun draf PO; hasilnya DRAFT (menunggu manajer) atau DITAHAN_GUARDRAIL.

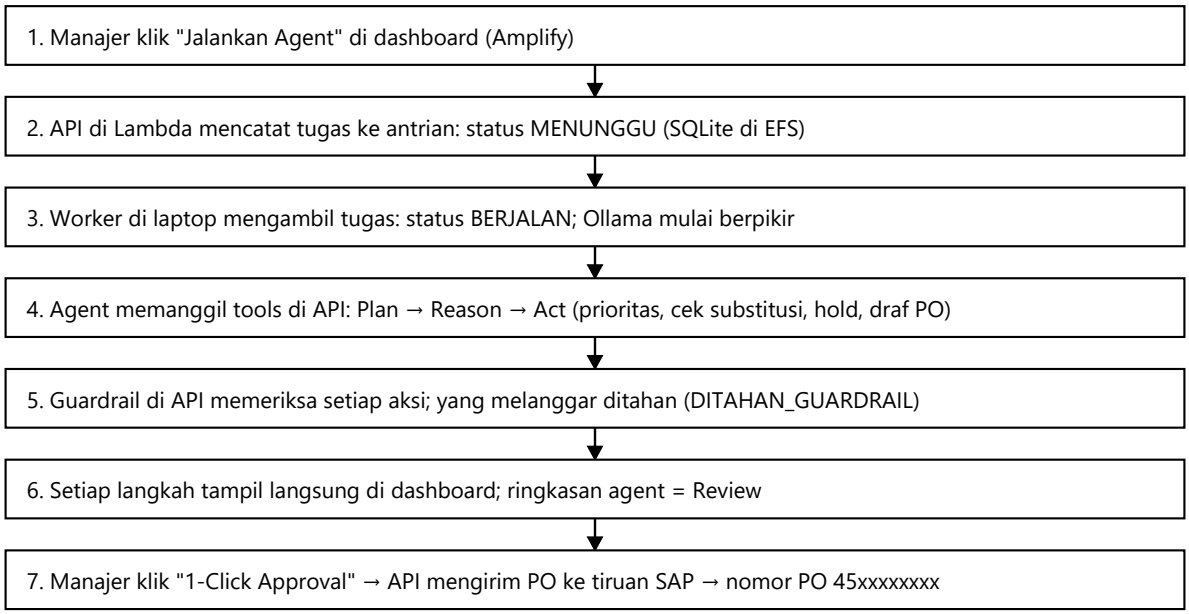
4. Dashboard ("yang dilihat manajer")

Komponen	Untuk apa
React + Vite + Tailwind	Membangun tampilan dashboard: panel agent (Plan → Reason → Act → Review), kartu PO menunggu persetujuan, tombol 1-Click Approval, Health Score, realokasi modal.
Kiro	AI coding editor dari AWS, dipakai tim untuk melanjutkan dashboard (memberi poin tambahan penilaian).

5. AWS ("tempat semuanya online")

Layanan	Untuk apa
AWS Lambda	Menjalankan API tanpa mengurus server; hanya aktif saat ada request sehingga murah. Function URL adalah alamat publiknya. Memori 1024 MB, container image Python 3.12.
Mangum	Adapter (penerjemah) supaya aplikasi FastAPI bisa berjalan di Lambda.
Docker + Amazon ECR	Docker membungkus API dan data DuckDB menjadi satu image; ECR menyimpan image itu di AWS.
Amazon EFS	"Hard disk bersama" untuk file SQLite. Tanpa EFS, data PO bisa hilang saat Lambda restart atau terbelah antar-salinan Lambda.
VPC + Security Group	Jaringan privat dan "satpam": Lambda boleh mengakses EFS, sedangkan EFS tertutup untuk pihak lain.
AWS Amplify	Hosting dashboard: https://main.d30b19xkk8lknt.amplifyapp.com
CloudWatch Logs	Catatan aktivitas Lambda untuk mencari error dan mengukur kecepatan.
AWS CLI + skrip PowerShell	Alat deploy: `deploy_lambda.ps1` (API) dan `deploy_dashboard.ps1` (dashboard).

6. Alur satu klik (yang ditunjukkan saat demo)



Langkah 2–6 adalah loop agentic yang diminta juri (plan, reason, act/tool, review). Langkah 7 adalah human-in-the-loop: tanpa persetujuan manajer, tidak ada PO yang masuk SAP.

7. Kenapa LLM berjalan di laptop, bukan di AWS?

Rencana awal memakai Amazon Bedrock. Di akun AWS tim (tipe proyek), kuota token harian Bedrock untuk semua model Amazon Nova bernilai 0 dan tidak bisa dinaikkan, dan pembuatan akun AWS standar gagal karena kartu ditolak saat verifikasi. Karena itu tim memilih: semua bagian lain di AWS, dan LLM di laptop lewat worker. Mentor AWS mengizinkan sebagian demo berjalan lokal. Kode agent tidak terikat satu penyedia model, sehingga LLM bisa dipindah ke Bedrock tanpa mengubah dashboard maupun API.

8. Perintah penting

Perintah	Fungsi
`python -m engine.pipeline`	Membangun ulang data (Kaggle → DuckDB) di laptop
`python -m agent.worker`	Menyalakan worker agent di laptop (wajib saat demo online)
`python -m agent.reset_demo`	Mengosongkan data demo online (PO, hold, jejak agent)
`deploy\deploy_lambda.ps1`	Deploy ulang API ke Lambda
`deploy\deploy_dashboard.ps1`	Deploy ulang dashboard ke Amplify